Speaking script — WireGuard for Apache NuttX (Community Over Code, Glasgow)

Deck: `coc-glasgow-slides.html` (and the version on the official template, `coc-glasgow-reveal/`), 40 slides, English. This file is **made by** `export_en_script.py` from the speaker notes in the deck (press `N` there to see them). The "Say" text below *is* those notes. Do not edit this file by hand. Change the notes in the deck and run the script again, or the two will stop matching. The Japanese practice script uses the same slide order.

Total: about 31:32, including a 90-second demo. The times add up from the start ("you should be at about this point"), at 150 words per minute. This is just over a 30-minute slot, so practice to finish the main talk in about 28 minutes, and keep the cut list below ready.

The most important part is slides 29–31 (testing). Together they take 4.1 minutes of the talk, and they hold the claims that a reviewer will really check. Their slides and narration were made shorter on 2026-10-12 (the detail is in the Q&A file and on GitHub). Do not cut their claims or their caveats (FLAT on boards / KERNEL and PROTECTED on QEMU only / sim, the earlier board runs vs. the 2026-10-12 ESP32-S3 run of the exact series, the PROTECTED wg stack fault and its fix, one scripted 4-CPU run, nothing shown on hardware for PROTECTED, the 75 s watch) any further. Cut before them or after them. They are one story: the first is where it runs, the second is what "it runs" does not prove, and the third is the one place where I had to decide instead of test. If you split them, the story breaks.

Do not cut slide 2 or slides 4–15 either (About Me, then Overview to Architecture). What NuttX is, where it is used, Sony's history with it, how I met it, and why that led to WireGuard: everything after that depends on this. Keep slide 37 (What I learned) too: it is the speaker's own message.

If time is short, cut in this order:

1. Slide 36 (Giving Back). It helps the talk, but the talk does not depend on it. If you cut it, answer it in Q&A if someone asks.
2. Slide 24 (the version 1 video still). The live demo shows the same kind of thing.
3. Slide 23 (Bug 3/6, the 50 ms polling). Its one-line lesson is already in the bug table.

If the live demo stops, switch to the recording right away. **The agenda slides (4, 11, 16, 19, 25, 32, 35) are a ten-second pause each**: point at the highlighted line and pause. Do not read the list. If you have no time at all, skip them without a word. They save almost no time, so cut the three slides above instead.

Check before the talk: the GitHub star numbers on slide 8 change. Check them again on the day, or keep the date on the slide.

---

1 — Title (0:00 → 0:50)

On slide: "WireGuard for Apache NuttX." ESP32-S3 & SPRESENSE tunnel over real Wi-Fi · real WireGuard peers (Linux kernel, Windows client) · runs in a kernel build on QEMU (BUILD_KERNEL) · one source across NuttX 13.0.1 / master / 12.7.0.

Say:

- Today I will talk about adding WireGuard to Apache NuttX.
- It is a WireGuard VPN, a virtual private network, that works as a network device called wg0. The tunnel runs over real Wi-Fi on ESP32-S3 and on SPRESENSE, and the other side is always a real WireGuard program — the Linux kernel module and the official Windows client. The apps version builds from one source on three NuttX versions. Later in the talk, you will see the new driver running inside KERNEL and PROTECTED builds on QEMU, a machine emulator, and in normal FLAT builds on both boards. That is the harder and more interesting half.
- This talk is about what "it works" was hiding, and what moving it into the kernel brought out.

2 — About Me (0:50 → 1:44)

On slide: Three bands: photo, name line and the fox icon / five hobby photos (space, semiconductors, DNA nanotech, robots, Linux) with three activity chips / a one-row path Robotics → Computer Vision → Edge AI → Embedded Systems → NuttX / WireGuard, a picture per step.

Say:

- First, a little about me.
- I am an engineer at Sony Semiconductor Solutions, working on edge AI: artificial intelligence that runs on the device itself. I work with Apache NuttX from the application side. I am from Japan: born in Nagoya, university in Sendai, and now I live in Kawasaki. Outside work, I am a hobby engineer. I build things around space, semiconductors and robots, I made DNA nanostructures at university, and I love Linux. On social media, I am the fox in this picture. I often join hackathons, I help run software conferences, and I wrote a book about open source.
- My road to this project went through molecular robots, computer vision and AI systems, edge AI cameras, and embedded systems on SPRESENSE. NuttX and WireGuard are the latest stop on that road.

3 — Agenda (1:44 → 2:23)

On slide: The seven parts, each a short title with a keyword line and its time: overview & background (~5 min) / the problem and the idea (~4) / the plan (~2) / version 1: the app, six bugs, two in detail (~4) / version 2: the kernel driver, a bug in NuttX, testing (~7) / live demo (~3) / summary & status, what I learned (~4).

Say:

- Here is the plan for this talk.
- There are seven parts. First, an overview and the background: what NuttX is, and how this started. Second, the problem I wanted to solve, and the idea. Third, the plan. Fourth, version one, an app: the first tunnel, the bugs it hid, and the first demo. Fifth, version two, the kernel driver: why I moved it, a bug in NuttX itself, and how I tested it. Sixth, a live demo with the StackChan. And last, where the work stands, what I learned, and a summary. About thirty minutes, then questions.

4 — Agenda: ① Overview and Background (2:23 → 2:33)

Say (one line): Let's start with part one: an overview and the background.

Pause: point at the highlighted line, one beat.

5 — Project Overview (2:33 → 3:23)

On slide: The wg0 netdev picture: application / NSH, the NuttX network stack, wg0 next to eth0 / wlan0, a UDP socket underneath, the Internet, the peer.

Say:

- Here is the whole idea in one picture.
- Apache NuttX is an RTOS, a real-time operating system. It follows POSIX, the Portable Operating System Interface, and it has its own TCP/IP stack: Transmission Control Protocol and Internet Protocol. It had no VPN. This project adds wg0 as a virtual network interface. Applications see a normal interface. Under it, the encrypted data travels over a UDP socket: User Datagram Protocol. The other side is a real WireGuard peer — Linux, Windows, or another gateway.
- Because NuttX follows POSIX and uses BSD sockets, the Berkeley style of sockets, it is a good fit for this work. FreeRTOS and Zephyr can run a VPN too. But on NuttX, a VPN can become a normal network device very naturally.

6 — How It Started: February to October (3:23 → 4:15)

On slide: A month axis Feb → Oct. Above it: mid Feb, read the GSoC idea list / early Mar, proposed the plan / Mar 20, heard about this conference from Alan, sent a talk / early Apr, GSoC proposal / early May, talk accepted, GSoC not selected / Oct, Glasgow. Below it: Jun–Jul a little at a time, August holiday most of the work, September FLAT app → kernel driver.

Say:

- Here is how this project went, month by month.
- In the middle of February, I read the GSoC idea list and started to look into it. In early March, I wrote my plan as an issue and proposed it. On March 20, Alan told me about this conference, so I sent a talk proposal. In early April, I sent my GSoC proposal.
- In early May, I got two answers. The talk was accepted, and the GSoC proposal was not. I decided to build it anyway.
- From June, I worked on it a little at a time. I did most of the work during the summer holiday in August. In September, I moved it from a FLAT app into a kernel driver. That is the second half of this talk.

7 — Background: What Is NuttX (4:15 → 5:07)

On slide: Software stack (applications / POSIX, ANSI, BSD socket API / networking, VFS, file systems, device drivers, NSH / NuttX kernel / MCU, MPU) and tags: RTOS, POSIX-oriented, BSD sockets, VFS, NSH, native networking, device drivers, ASF top-level project.

Say:

- First, what is NuttX?
- NuttX is a real-time operating system that puts standards first. It follows the POSIX and ANSI APIs, that is, the standard application programming interfaces, it uses BSD sockets for networking, and it has a real file system, a shell called NSH, the NuttShell, built-in networking and device drivers. All of this is made small enough to run on microcontrollers, from 8-bit to 64-bit. It is also a top-level project of the Apache Software Foundation, and its community develops it in the open.
- The one point I want you to remember: NuttX is a small RTOS, but you program it like Unix. That is rare. And that is why almost everything later in this talk — sockets, network devices, a POSIX-style port — is possible at all.

8 — Where NuttX Fits (5:07 → 6:17)

On slide: FreeRTOS / Zephyr / NuttX table (design centre, typical use), GitHub stars with the note that stars are visibility, not deployment; number cards: 15+ architectures, 300+ boards, 1500+ configs, Sony, PX4, Xiaomi OpenVela 1000+ SKUs, Japan's 2024 lunar mission.

Say:

- Thirty seconds on where NuttX fits, and whether it is a toy.
- These are three good RTOSes, and each has a different focus. FreeRTOS is a small real-time kernel, on purpose, for MCU firmware, that is, microcontroller firmware, and control. Zephyr is an all-in-one platform for connected devices, wearables and industrial IoT, the Internet of Things. NuttX is the Unix-like one: networking, file systems and drivers, used in drones and sensing. This is not a ranking. By GitHub stars, NuttX is the smallest of the three, and I want to say that clearly. But stars show how many developers notice a project, not how many devices use it.
- And it is really used. Upstream, NuttX supports more than fifteen CPU, or processor, architectures, more than three hundred boards, and more than fifteen hundred configurations. In real products it runs in Sony devices, PX4 flight controllers, and more than a thousand Xiaomi OpenVela products, and it flew on Japan's 2024 Moon mission. It is small next to the big ones, but it is not a toy.

Check before the talk: the star counts move. Re-check them on the day, or leave the date on the slide as it is.

9 — History: NuttX at Sony (6:17 → 7:19)

On slide: Timeline: 2015 Sony audio products / 2018–19 SPRESENSE, CXD56xx (OSS + ELC Europe 2019) / 2020 upstream collaboration (NuttX Online Workshop, "A Journey from Fork to Mainline") / 2020s sensing and edge devices.

Say:

- This is Sony's history with NuttX. It is not my story yet, and it is older than edge AI.
- Sony has sold audio products running NuttX since around 2015. A few years later, SPRESENSE and its CXD56xx chip added audio, GNSS satellite positioning, camera and multi-core support to NuttX. In 2019, Masayuki Ishikawa from Sony presented this work at Open Source Summit and Embedded Linux Conference Europe. After that, Sony engineers became major upstream contributors. One of them, Alin Jerpelea, later chaired the NuttX project management committee. His 2020 talk was called “A Journey from Fork to Mainline.”
- I am not saying that one engineer made Sony choose NuttX. I have no source for that, so I will not say it. What public talks and documents do show is simpler: Sony used NuttX for many years before edge AI, and over time it spread into camera, GNSS and sensing work. My own work builds on that.

10 — Experience: SPRESENSE and AITRIOS (7:19 → 8:19)

On slide: Photos of SPRESENSE and the AITRIOS edge AI camera; the flow "application development → system integration → deployment & operation".

Say:

- That was Sony's history. This is mine. Same hardware family, but I see it from a different place.
- On SPRESENSE, I use the NuttX SDK, the software development kit, for sensing, camera, GNSS and LTE mobile networks — prototypes and embedded development, on the application side. On AITRIOS edge AI cameras, which are built with ESP32 and NuttX, my job is to set up cameras and keep them running on site, not on a desk next to a debugger.
- In both cases, I come to NuttX from the same direction: first application development, then putting the system together, then setting it up and running it. I never had to write a NuttX driver for any of that — until this project. And the question that brought me here was not “How do I make NuttX run?” It was “How do I reach a NuttX device after it is installed?”

11 — Agenda: ② The Problem and the Idea (8:19 → 8:29)

Say (one line): That is the background. Part two: the problem I wanted to solve, and the idea.

Pause: point at the highlighted line, one beat.

12 — Problem: Reaching a Device From Far Away (8:29 → 9:22)

On slide: Engineer → Internet → router → Wi-Fi → NuttX device, showing that WPA2/WPA3 covers only the Wi-Fi hop; cards: open it to the Internet / make your own protocol / use a vendor's cloud.

Say:

- Now, the problem itself.
- Wi-Fi security is real, but it protects only one part of the path: the radio link between the device and the access point. Everything before that — the engineer, the Internet, the router — is outside what

WPA2 or WPA3, the Wi-Fi security standards, can protect. What I needed was a secure path for the whole way, from the engineer to the device, not just the last few meters.

- Without that, every choice has a problem. You can give the device a public IP and hope. You can make your own secure protocol and keep it working forever. Or you can hand the connection to a vendor's cloud. The same problem appears in edge AI, industrial IoT, and remote systems in general. It is not limited to this project.

13 — Options: Other Ways to Reach a Device (9:22 → 10:44)

On slide: A table of seven ways (open a port / HTTPS to a cloud API / MQTT broker / reverse tunnel or relay / VPN on the router / carrier closed network / VPN on the device) against: who connects, any service or not, server in the middle, where security ends, what NuttX already has.

Say:

- Before I chose WireGuard, I compared the usual ways to reach a device.
- Opening a port is easy, but then the whole Internet can reach the device. HTTPS, secure web requests, and MQTT, a small publish-and-subscribe messaging protocol, are what most IoT products use. The device connects out to a server, so NAT, network address translation, is not a problem, and NuttX already has clients for both. But you only get the messages you designed — you cannot log in or open a web page on the device. And the security ends at the broker or the cloud.
- A reverse tunnel gives you one port through a relay server. A VPN on the router is great when the router is yours. But the last part of the path, on the LAN, the local network, is not protected. And at a customer's site, the router is usually not yours. A carrier closed network works only over cellular, and you need a contract.
- A VPN on the device is the only row that has all three: any IP service, no server in the middle, and security all the way to the device. MQTT and HTTPS are still the right tools for data. This is for maintenance access.

If asked about Tailscale or ZeroTier: they are also a VPN on the device, but they usually need a bigger OS than a microcontroller has today (footnote on the slide).

14 — Solution: Why WireGuard (10:44 → 11:37)

On slide: Engineer / Linux / Windows ↔ WireGuard tunnel ↔ NuttX device; modern IP-layer VPN / UDP transport, public-key peers / already everywhere.

Say:

- So, why WireGuard?
- WireGuard is a modern VPN that works at the IP layer. It is small and simple by design, with much less code than IPsec or OpenVPN, and that matters a lot on a microcontroller. It runs over plain UDP, and each peer is known by its public key, not by certificates or a long setup. If a packet does not carry the right key, WireGuard does not answer at all — to everyone else, the device is just a brick wall. And most importantly, it is a standard. It has been in the Linux kernel for years, there is an official Windows client, and the other end can be any WireGuard peer.

- So I chose a standard protocol, instead of making my own or depending on a vendor's cloud.

15 — Architecture: WireGuard on the NuttX Network Stack (11:37 → 12:25)

On slide: Application → TCP/UDP/IP → wg0 (NET_LL_TUN netdev) → WireGuard encrypt/decrypt → UDP socket → wlan0 / usrsock → network, with TX and RX arrows.

Say:

- This is one step more technical than the overview. This is where the implementation starts.
- Going out, a packet comes down from the application through TCP or UDP and IP, and reaches wg0 like any other network device. WireGuard encrypts it, and it leaves as a UDP packet — through wlan0 on a board, or through usrsock when the Wi-Fi chip itself works through sockets. Coming in is the same path the other way: a UDP packet arrives, WireGuard decrypts it, and it is passed up to wg0 as if it came in on a normal interface.
- From here on, the work is building this middle part — wg0 and the encryption — without changing the layers above or below it.

16 — Agenda: ③ The Plan (12:25 → 12:35)

Say (one line): Part three: the plan. How do you reuse an existing WireGuard?

Pause: point at the highlighted line, one beat.

17 — Porting Strategy: Reusing wireguard-lwip Without lwIP (12:35 → 13:39)

On slide: Three cards: protocol & crypto (KEEP) / platform hooks (ADAPT, the four functions in wireguard-platform.h) / lwIP network glue (REPLACE).

Say:

- Why I started from wireguard-lwip, an existing open-source WireGuard, and did not write the crypto myself.
- wireguard-lwip is cleanly split into three layers. The protocol and crypto — the handshake, the data transport, BLAKE2s, ChaCha20-Poly1305 and Curve25519 — are portable C that does not depend on any OS, any operating system. I used the crypto without changes. The protocol core needed two small fixes later, after a review, and they are written down in PATCHES.md. Everything that depends on the OS was already kept behind one small header file, as four functions. I only had to replace the lwIP network glue — the part that registers an lwIP netif and handles lwIP buffers — because NuttX does not have lwIP at all.
- I did not have to rewrite the protocol or the crypto, and that was not luck. That is what a clean line between the protocol and the OS gives you. Because of it, this port took weeks, not months.

18 — Design: Mapping lwIP onto NuttX netdev (13:39 → 14:08)

On slide: netif → net_driver_s, pbuf → iob, netif_add() → netdev_register(), callbacks → devif_poll().

Say:

- This was the most important porting decision.
- Instead of bringing lwIP into NuttX, I matched each lwIP idea to NuttX's own. lwIP's network interface becomes a NuttX network driver, lwIP's packet buffers become NuttX's I/O buffers, adding an interface becomes registering a NuttX network device, and the send callback becomes NuttX's own poll. wg0 is registered as a TUN-type, or tunnel-type, network device, and the “wire” under it is a UDP socket.

19 — Agenda: ④ Version 1: The App (14:08 → 14:18)

Say (one line): Part four: version one, an app. It worked, and then the bugs came.

Pause: point at the highlighted line, one beat.

20 — Result: A Working Tunnel Is Only the Beginning (14:18 → 15:03)

On slide: Protocol & crypto: crypto byte-identical, protocol core with two small fixes (PATCHES.md) / OS hooks: four functions / network glue: the real work. wg genkey / set / setconf at run time.

Say:

- Here is the result of the first version.
- More than three thousand lines of protocol and crypto came over. The crypto came over exactly as it was. The protocol core needed two small fixes after a review — and one of them was a bug in wireguard-lwip itself: its rate limit never worked. The four OS functions are small. The real work was the network glue. And it worked: you can make a key, set peers and load a config, all at run time, with no rebuild. So far, a success story — but the princess was in another castle. The rest of this talk is about what that success was hiding.

21 — Where It Was Hard: Six Bugs (15:03 → 16:12)

On slide: One table of all six bugs (bug / showed up in / lesson): 1 the receive thread stopped with the wg command / 2 ping works, TCP dies (in detail) / 3 "no real impact" was 10× (in detail) / 4 stack 94% full, saving the config froze the board, real boards only / 5 NuttX's own ChaCha20-Poly1305 (part 5) / 6 a 6 KB array on a 3 KB kernel stack (part 5). The GitHub line with a QR code. Takeaway: most hid behind a test that passed.

Say:

- Now, where it was hard.

- The first tunnel worked quickly. Most of my time went into six bugs: four in the apps version, and two more after I moved it into the kernel. Most of them hid behind a test that passed, like a handshake or a ping, so I first looked in the wrong place.
- Bug one: the receive thread was created as detached, and it stopped when the wg command that started it exited. Detach only decides when the memory is freed, not how long the thread lives. That is why, in the kernel version, the driver owns its thread. Bug four: only real hardware showed it. On the board, the receive task's stack was ninety-four percent full, with only 176 bytes left. Sim has big stacks, so it never saw it. Please remember this one.
- I will tell two of them in detail, because their lessons are useful for anyone. Bugs five and six come in part five. The logs and fixes for all six are in the GitHub repository.

Bugs 1 and 4 have no slide of their own any more. Their story is in the notes above; the full version (and bug 1's `SO_RCVTIMEO` detour) is in the Q&A file.

22 — Bug 2/6: ping Works, TCP Dies (16:12 → 17:07) *(my favorite)*

Say:

- This is my favorite bug in the project.
- On the same tunnel, ping was perfect, and even a TCP handshake finished — and then the connection died. When I added logging to the send path, it showed EBADF: bad file descriptor. All the sends that worked happened inside the receive task itself — handshake replies, keepalives, ping replies, the SYN-ACK, the second step of a TCP connection. Sends from any other task failed.
- The reason: in NuttX, a file descriptor belongs to one task group. It is not shared by the whole system. The fix was to stop using file descriptors and use NuttX's internal socket API. It works on a struct socket, which is just memory, so any task can use it. That is exactly why the kernel version keeps the socket as a struct socket.

23 — Bug 3/6: "No Real Impact" Was 10× (17:07 → 17:48) *(cut candidate 3)*

Say:

- The third: something I never measured.
- The fix without file descriptors also broke poll, because poll works with file descriptors. So, as a quick fix, I checked the socket every fifty milliseconds. I thought the only cost was waking up the CPU when it was idle. I was wrong. Every packet that arrived waited up to fifty milliseconds, and that alone limited the speed — about ten times slower than it should have been. I thought it had no real cost, and it cost ten times. The real fix was a proper wait: the socket wakes the thread up when data arrives.

24 — Version 1 Demo: The First Video (17:48 → 18:14) *(cut candidate 2)*

On slide: Left: a still frame of the video (the Windows WireGuard client beside a telnet session into NuttX on the ESP32-S3). Right: the steps telnet 10.10.0.2 / uname -a, ifconfig / webserver & / browser

http://10.10.0.2/; ESP32-S3 DevKit, apps version (FLAT), USB unplugged; the official Windows client; small text youtu.be/1kyX2av5WG4.

Say:

- Here is a frame from the version one video: I logged in with telnet through the tunnel, ran a few commands, and opened a small web page on the board. The board runs on a power adapter only, and the other end is the official Windows WireGuard client. So version one was enough for a demo; next, I made it a real NuttX driver.

No playback. The still is enough; the video is the backup for the live demo.

It is the apps version. Do not mix it with the kernel-version results later in the talk.

25 — Agenda: ⑤ Version 2: The Kernel Driver (18:14 → 18:24)

Say (one line): That was version one. Part five, the harder half: a real kernel driver, and how I checked it.

Pause: point at the highlighted line, one beat.

26 — Why the Kernel: From App to Driver (18:24 → 19:12)

On slide: Two panels: Version 1, an app (in apps/, uses kernel internals, FLAT only, never an in-tree driver) / Version 2, a kernel driver (drivers/net/wireguard, set up by a small wg command through ioctl, made for FLAT, PROTECTED and KERNEL). One line on the three build modes.

Say:

- Why did I move it into the kernel?
- Version one is an app. It lives in apps and reaches into socket internals of the kernel. That works in a FLAT build, where everything shares one memory space. But it can never be a proper in-tree driver, and it cannot work in PROTECTED or KERNEL builds, where apps and the kernel are kept apart.
- So I moved the device into the kernel, with the other network drivers, and a small wg command sets it up through ioctl, the input/output control call. Version one showed that the protocol works on NuttX. Version two is about fitting NuttX itself: a normal device, in every build mode, in a shape the community can review.

27 — The Kernel Design: The Big Picture (19:12 → 19:58)

On slide: A picture: user space holds only the wg command; an ioctl (one flat struct, no pointers; the private key is write-only) crosses the user/kernel boundary to wg0 in the kernel, which owns its UDP socket and RX thread, keeps timers / peers / cookies and uses NuttX crypto, between the NuttX network stack and the encrypted UDP to the peer.

Say:

- This is the shape of the real driver.

- In user space, there is only a small wg command. It sends settings to the kernel through ioctl. Each ioctl carries one flat struct with no pointers inside, so in PROTECTED and KERNEL builds the kernel never has to follow a pointer into user memory. The private key is write-only: no ioctl ever gives it back.
- In the kernel, wg0 is a normal network device. It owns its UDP socket and its receive thread, so the thread and socket bugs from version one cannot happen, by design. The crypto is NuttX's own. The protocol core is still derived from wireguard-lwip under BSD-3-Clause, and the LICENSE file says so.

28 — Bugs 5 & 6/6: Two More Bugs, One in NuttX Itself (19:58 → 21:05) *(locked with the kernel design)*

On slide: Two panels: NuttX's own chacha20poly1305 nonce bug (handshake works, data after the first packet fails; two-line crypto: fix) / a BUILD_KERNEL-only crash (6 KB array on a 3 KB kernel stack, moved to the heap). One line: SPRESENSE did not boot with CONFIG_RTC_HIRES on 13.0.1 / master; fixed locally, report drafted, not sent.

Say:

- On the way into the kernel, two more bugs came, and one of them was in NuttX itself.
- First, NuttX's own ChaCha20-Poly1305 wrote the counter into the wrong bytes of the nonce. When the counter is zero, both layouts are the same, so the handshake works, and then data packets after the first one fail to decrypt. Nothing in NuttX had called this code before, so nobody had hit it. It is a two-line fix, kept as its own crypto change.
- Second, a crash only in a kernel build. Setting the peers put a six-kilobyte array on a three-kilobyte kernel stack. In the simulator, with big stacks, it never showed up. It is the same stack lesson as bug four, one level deeper. I moved the array to the heap.
- One more from the board side: on NuttX 13.0.1 and master, SPRESENSE did not boot with the high-resolution RTC, real-time clock, option. I found the cause and fixed it locally. The report is drafted, but not sent.

29 — Testing: What Ran Where (21:05 → 22:48) *(core — do not cut)*

On slide: Three cards. FLAT on real boards: ESP32-S3 2026-10-12 with the exact submitted series (handshake with Linux, ping both ways up to the full MTU) / an earlier driver build on ESP32-S3 + SPRESENSE to the Windows client / SPRESENSE memory after 40 down/up. KERNEL on QEMU: tunnel to Linux, 4-CPU stress, one scripted run. PROTECTED on QEMU: tunnel works, the wg stack overflow found and fixed (20/20). Then "Not shown yet: PROTECTED or KERNEL on real hardware" and the GitHub line with a QR code.

Say:

- Let me be exact about which version ran where. There are three build modes, and I keep them apart.
- FLAT, on real boards. On October twelfth, I ran the exact series I prepared for upstream on the ESP32-S3 inside the StackChan: a handshake with Linux WireGuard, and ping both ways, up to the full packet size. An earlier build of the driver ran on both boards over real Wi-Fi with the Windows

client. On SPRESENSE I measured memory: after forty down-and-up cycles, it was exactly the same. The receive thread used fourteen hundred and seventy-two bytes of a six-kilobyte default. That default used to be a guess. Now it is a measurement.

- KERNEL, in QEMU. The wg command is a separate program that reaches the driver through system calls, and the tunnel reaches Linux. With four CPUs, I flooded it while changing the settings. No fault, but that is one scripted run, not a full search for race conditions.
- PROTECTED, also in QEMU. The tunnel works, and every ioctl crosses the memory boundary. Repeated runs sometimes faulted. The cause was a stack overflow in the wg command, because in this mode a system call runs on the caller's own stack. A bigger default fixed it: twenty out of twenty.
- What I have not shown is PROTECTED or KERNEL on real hardware. I tried PROTECTED on the CoreS3, but that board does not boot any bootloader-based image, and that is not a WireGuard problem. Version one, the app, also ran on both boards. The records are on GitHub.

30 — Testing: What Ping Misses (22:48 → 23:46) *(core — do not cut)*

On slide: Three cards: same key on every boot? (SPRESENSE, 3 resets, 3 keys) / keys left after down? (sim, all zero) / a leak after hours? (sim, 151 rekeys, 100 down/up, nothing lost).

Say:

- This slide is about what a working tunnel cannot tell you: three problems that ping would never notice.
- First, keys. NuttX can seed its random device from build-time settings. Then a board makes the same private key every time it starts, and everything still works. So I reset a SPRESENSE three times and got three different keys. That is a quick check, not a proof of good randomness.
- Second, I read the device's memory in the simulator. After down, the session keys were all zero. The static key stays on purpose, so that up works again.
- Third, a long test, also in the simulator: a hundred and fifty-one forced rekeys and a hundred down-and-up cycles. No buffer was lost, and memory went back to exactly the start-up number. I am not saying it ran for a week, or that the simulator acts like the board.

31 — A Design Choice: The Timestamp Problem Has No Easy Answer (23:46 → 25:09) *(core — do not cut)*

On slide: The rule (a newer timestamp, even after a reboot), Chosen (real-time clock + the highest value this boot, plus a warning) / Not chosen (save every time, reserve a range in flash, wait), and the measured pair: clock unset → no handshake in 75 s, clock set → 4.1 s.

Say:

- This is the one place where I had to decide, not test. And then I measured my decision.
- WireGuard's replay protection has a rule: every handshake needs a newer timestamp than the peer has seen, even after a reboot. The device is usually the side that starts the handshake, and it is also the side least likely to have a clock with a battery.

- Saving every timestamp wears out the flash. Reserving a range in flash is the right long-term answer, but it needs a write guarantee I could not verify on these file systems. And just waiting means a device that ran for a month cannot reconnect for a month. Here, the easy choice and the right choice were not the same.
- So I chose the real-time clock plus the highest value used in this boot, exactly what Linux and wireguard-go do. And the driver warns once if the clock was never set.
- Then I measured it on the board. I sent the board nothing at all, because a ping from the peer would start the handshake for it. With the clock as it boots: no handshake in a seventy-five-second watch. With the clock set: four point one seconds. That is why the driver prints a warning.

32 — Agenda: ⑥ Live Demo (25:09 → 25:19)

Say (one line): That was all engineering. Part six: what it actually looks like.

Pause: point at the highlighted line, one beat.

33 — Demo Setup: One Tunnel on a Phone's Network (25:19 → 26:17)

On slide: Diagram: venue Wi-Fi (blocks devices from talking to each other) → phone hotspot (its own small local network) → laptop (WireGuard peer in Docker, wg0 10.10.0.1; voice clips on :8000) and StackChan (ESP32-S3 + NuttX, wg0 10.10.0.2 over wlan0 on the hotspot; apps version v0.1.1, FLAT build), joined by one WireGuard tunnel; SPRESENSE as a dashed second peer (wg0 10.11.0.2).

Say:

- Here is the demo setup.
- My laptop and the StackChan are both on my phone's hotspot. The venue Wi-Fi works, but it blocks devices from talking to each other, so the phone makes a small local network for us. The phone can use the venue Wi-Fi or mobile data for its own Internet, but the demo does not need it.
- Between the laptop and the StackChan there is one WireGuard tunnel. The laptop side is a normal Linux WireGuard in a container, at 10.10.0.1. The StackChan is 10.10.0.2. Everything in the demo — telnet, the web page, and even the voice — goes through that tunnel. I made the voice clips on the laptop in advance, so there is no cloud API anywhere.
- Please watch two things: the addresses after I log in, and the packets. Outside the tunnel, you only see encrypted UDP.

34 — The Demo (26:17 → 27:47)

On slide: Seven steps: wg show / telnet 10.10.0.2 / ifconfig / stackchan face happy / stackchan say hello.wav / tcpdump outside vs. inside / wg show again. Right panel: outside = only encrypted UDP 51820, inside = the telnet text; "Runs the apps version (v0.1.1), FLAT build."

Say:

- The point of this demo is that there is nothing special to see. The StackChan is just a normal computer at the far end of an encrypted tunnel. This board runs the apps version — the same app as in the video. The kernel driver ran on this chip too, as you saw in the testing part, but the robot demo is built on the app.
- First, wg show on the laptop: the handshake is done, and you can see the StackChan's address on the phone's network. Then I telnet to 10.10.0.2 — that is the address inside the tunnel. When I type ifconfig, you see both: wlan0 on the phone's network, and wg0 inside the tunnel. So I am really logged in through the tunnel.
- Now I can control it. Let's make it happy, and let it say hello. The voice comes from the laptop through the same tunnel.
- Last, the packets. On the left, the phone's network: only encrypted UDP. On the right, inside the tunnel: the same telnet session, in plain text. And wg show again: the byte counts went up. Mischief managed.

Live: `scripts/stackchan/demo.sh`: `login` in the left terminal, `outside` and `inside` in two terminals on the right, `show` before and after. Put the terminals on the left and the StackChan (camera or on the desk) on the right.

Before the talk: phone hotspot on (2.4 GHz / maximize compatibility), laptop on it, Docker Desktop running, the StackChan joined with `nsh_wifi.py` (the passphrase is not saved on the board), `wg_setup.py` run once. Check `stackchan say` once. If the hotspot was turned off and on, its addresses may change: run `nsh_wifi.py` and `wg_setup.py` again. Rarely (about 1 in 20) the board stops during `say`; power it off and on, then repeat the same steps.

Never on screen: `wg showconf` (prints the private key), the key files in `%USERPROFILE%\stackchan-wg`, the hotspot passphrase.

Fallback: the recording. Switch at once if the live demo stalls.

35 — Agenda: ⑦ Summary and Status (27:47 → 27:57)

Say (one line): Last part: where the work stands, what I learned, and what to take home.

Pause: point at the highlighted line, one beat.

36 — Giving Back: Where It Stands (27:57 → 28:39) *(cut first)*

Say:

- Here is where the work stands.
- It is split into small pieces: a small crypto fix that is useful on its own, the driver for the kernel device, and the wg command for user space. The PROTECTED stack fix is in the command's default settings. Every commit passes the patch checks and builds on its own, and the exact series runs on the ESP32-S3 in a normal FLAT build. Nothing is sent yet. Each commit still needs my sign-off and an Assisted-by line, because AI tools helped me. Community over code means being honest about where the work really is, and then letting review change it.

If asked "is it merged?": not yet. These are submission candidates, not a claim that it is mergeable. See the Q&A file.

If time is short: skip the whole slide. The talk's argument (the pattern, the verification) does not rest on it; just keep the answer above ready for questions.

37 — What I Learned (28:39 → 29:59)

On slide: Three cards: from application user to OS contributor / AI helped me code, but understanding was my job (highlighted) / a welcoming open-source community (docs and guides, review, Assisted-by and no AI Signed-off-by, CI and hackathons). Bottom line: the hope to keep contributing.

Say:

- Before the summary, three things I learned.
- First, I came to NuttX as an application user. This project took me into drivers, the kernel, and even bugs in NuttX itself.
- Second, and I want to say this one clearly. I used AI, Claude, a lot in this project. It made research and writing code much faster. But embedded OS internals were new to me, and really understanding the design and the behavior was hard. I could not just take the code the AI wrote. I had to ask questions, read the NuttX source, test the behavior, and keep going until I could explain it myself. AI helped me code, but understanding was my job.
- Third, the community. NuttX has good documentation and contributing guides, and active technical discussion and review. It is open to AI tools, but people stay responsible: a commit says Assisted-by, and AI never adds a Signed-off-by. And CI, continuous integration, and hackathons help new people join. Open source is not only code that is public. It grows when new people can join, learn, and contribute.
- I hope to keep contributing to Apache NuttX and the open-source community, and I look forward to seeing NuttX continue to grow.

Say the AI part plainly. Not as a confession and not as a boast: it was faster, and understanding was still hard and still mine.

38 — Summary: What I Built, and Three Technical Lessons (29:59 → 30:47)

Say:

- To sum up.
- I built wg0, a WireGuard network device inside the NuttX kernel. It runs on the ESP32-S3 and SPRESENSE in normal FLAT builds, and in KERNEL and PROTECTED builds on QEMU. It is prepared as small PRs, pull requests, and nothing is sent upstream yet.
- Three technical lessons. First, test beyond the handshake: long data runs, real hardware, and every build mode. Second, a clean line between your code and the OS pays off. The apps version ran on four CPU types — x86, Cortex-A7, Xtensa and Cortex-M4F — and on three NuttX versions, with no code change. Third, porting to a project is a chance to make it better: one bug was in NuttX's own crypto.

39 — Thanks: The People Behind This Talk (30:47 → 31:22)

On slide: Four boxes: Alan / the Apache NuttX community / the Apache Software Foundation and the CoC organizers and volunteers / Sony's NuttX developers, past and present.

Say:

- Before I finish, some thanks.
- First, Alan, who told me about this conference and encouraged me to submit a talk. Second, the Apache NuttX community: when I wrote my GSoC proposal, people talked with me and encouraged me, and that is why I kept going after it was not selected. Third, the Apache Software Foundation and everyone who organizes Community Over Code. And last, the Sony engineers before me, who brought NuttX into our products and gave so much back to the project. This work stands on theirs.

40 — Thank You (31:22 → 31:32)

On slide: The GitHub repository with a QR code, the demo video link, questions welcome.

Say:

- Thank you. I am happy to take your questions — about the kernel driver, the bugs, the ioctl ABI, the binary interface, or the hardware.

After: keep coc-glasgow-qa.md at hand, not a slide. The usual questions (rebase status, #14, IOB exhaustion, checkpatch) are answered there and on no slide.